Robust Privacy-Preserving Federated Learning for Edge Computing With New Client Integration

Hao Zhou, *Member, IEEE*, Hua Dai, *Member, IEEE*, Geng Yang, and Yang Xiang, *Fellow, IEEE*

***Abstract*—Federated learning (FL) is a key paradigm for deploying AI models across large numbers of Internet of Things (IoT) devices in edge computing. While FL avoids uploading raw data to a central server, client privacy remains vulnerable during new client integration, when previously unseen devices first register their identities and cryptographic keys. A malicious or semi-honest central server (CS) can manipulate training to isolate target gradients, reconstruct local data, and tamper with aggregation. We study the Identity Forgery and Gradient Inversion Attack (IFGIA) against federated edge learning. By fabricating virtual clients and exploiting secure aggregation, a malicious CS can recover target gradients with success rates above 99.5% under realistic edge settings, revealing a critical weakness in existing privacy-preserving and verifiable FL schemes. To defend against IFGIA, we propose Robust Federated Learning (RFL), a framework tailored for edge computing that combines model splitting between edge clients and edge servers, lightweight differential privacy on intermediate representations, and split verification using digital signatures and homomorphic hashes. Experiments show that RFL reduces IFGIA's success rate to 73.6%, shrinks per-client communication from 375 MB to 40 KB, accelerates edge-side training by at least 10×, and maintains competitive accuracy.**

***Index Terms*—Federated learning (FL), edge computing, privacy preservation, verifiable aggregation.**

Received 25 March 2024; revised 8 December 2025; accepted 21 December 2025. Date of publication 6 January 2026; date of current version 12 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62372244 and Grant 62572253, in part by Jiangsu Provincial Natural Science Foundation (Youth Fund) under Grant BK20250683, in part by the Natural Science Foundation of Nanjing University of Posts and Telecommunications under Grant NY224058 and Grant NY225124, in part by Young Elite Scientists Sponsorship Program by Jiangsu Association for Science and Technology under Grant JSTJ-2025-641, in part by the Basic Science (Natural Science) Research Project of Higher Education Institutions in Jiangsu Province under Grant 25KJB520040, in part by the Open Project of State Key Laboratory for Novel Software Technology under Grant KFKT2025B68, in part by China Postdoctoral Special Grant Foundation under Grant 2024T170432, and in part by Chinese Government Scholarship, and Swinburne University through the Department of Defence Next Generation Technologies Fund, administered by the Advanced Strategic Capability Accelerator, in undertaking this work. *(Corresponding author: Hua Dai.)*

Hao Zhou is with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, China, also with the State Key Laboratory of Tibetan Intelligence, Nanjing 210003, China, and also with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210003, China (e-mail: haozhou@njupt.edu.cn).

Hua Dai and Geng Yang are with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, China, and also with the State Key Laboratory of Tibetan Intelligence, Nanjing 210003, China (e-mail: daihua@njupt.edu.cn; yangg@njupt.edu.cn).

Yang Xiang is with the Swinburne University of Technology, Melbourne 3122, Australia (e-mail: yxiang@swin.edu.au).

Digital Object Identifier 10.1109/TDSC.2026.3651107

I. Introduction

FEDERATED learning for edge computing has emerged as a prominent distributed paradigm for deploying deep learning models across large numbers of Internet of Things (IoT) devices [1], [2]. By allowing edge clients to collaboratively train models without uploading raw data to a centralized server, FL mitigates direct data exposure [3], [4], [5], [6]. However, the decentralized nature of FL also introduces serious privacy risks, especially in heterogeneous edge environments where many resource-constrained clients participate in collaborative model training [7].

In particular, the central server (CS) can manipulate the training process and exploit intermediate model updates to infer private information about clients from gradients or labels, and to tamper with aggregation results [8], [9], [10], [11], [12], [13], [14], [15]. This threat is particularly severe during new client integration, where an edge device participates in FL for the first time and establishes its cryptographic credentials and trust relations with edge servers (ESs) and the CS. This threat is particularly severe during new client integration, where an edge device participates in FL for the first time in a training round and establishes its cryptographic credentials and trust relations with the ESs and CS. Such joining phases are highly vulnerable to identity forgery and gradient manipulation by a malicious CS and thus require dedicated protection mechanisms.

In this context, recent efforts have focused on preventing privacy leakage and ensuring the integrity of aggregation models. However, as edge clients contribute more to the model during training, the risk of privacy leakage intensifies. Current strategies, including introducing artificial noise to global model parameters and locally encrypting and decrypting model parameters, have shown limitations, leading to diminished data availability and suboptimal model performance [16], [17], [18], [19], [20], [21], [22], [23]. Additionally, methods relying on secure multiple computing for privacy preservation have demonstrated vulnerabilities to attacks orchestrated by the central server [24], [25], [26].

In this paper, we rigorously examine the vulnerability of federated learning for edge computing to identity forgery and gradient inversion attacks against privacy preservation and verifiability. We demonstrate that current schemes are highly susceptible to IFGIA, with success rates exceeding 99.5%. To counter this threat, we propose a robust federated learning framework tailored for edge computing, ensuring privacy during multiple gradient releases and authenticating the identities of edge clients

1545-5971 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS

Symbol	Meaning
n	number of edge clients
u	index of edge client
p_u	intermediate parameters of client u
F_u	local loss function of client u
d_G	dimension of the global model
HH	homomorphic hash function
DS	digital signature scheme

and aggregation results' integrity. Our main contributions are summarized as follows:

- We identify and formalize the Identity Forgery and Gradient Inversion Attack (IFGIA) in federated edge learning, and show that existing secure-aggregation-based verifiable FL schemes are highly vulnerable to this attack.
- We design RFL, a robust federated learning framework tailored for edge computing, which jointly provides privacy preservation, client identity verification, and aggregation integrity.
- We introduce a lightweight differential privacy mechanism on intermediate representations, which reduces the success rate of IFGIA from 99.5% to 73.6% in our experiments.
- We propose a split verification mechanism based on digital signatures and homomorphic hashes, enabling edge servers to verify both client identities and the correctness of aggregation results.
- We employ model splitting and edge-side feature reuse to achieve dimension-independent computation and communication overhead at edge clients, accelerating client-side training by at least an order of magnitude and significantly reducing communication volume.

We focus on a three-tier federated edge learning architecture comprising a cloud-based central server (CS), multiple edge servers (ESs), and a large number of resource-constrained edge clients (ECs). The CS coordinates the overall training process and maintains the global model, while ESs act as trusted intermediaries that aggregate local updates from ECs within their coverage and relay aggregated results to the CS. ECs are end devices that locally train models on their private data. In our threat model, the CS can be semi-honest or malicious and may attempt to forge client identities and manipulate gradient information, whereas ESs and ECs are assumed to follow the protocol honestly. This layered architecture is typical in edge computing deployments and provides the practical context in which the proposed IFGIA and RFL framework are investigated.

The remainder of this paper is organized as follows: Section II provides background information on federated edge learning and global privacy data leakage prevention. Section III delves into the identity forgery and gradient inversion attack. Section IV outlines the system model and threat scenarios. Section V details the proposed RFL framework. Section VI conducts a thorough analysis of RFL's security and efficiency aspects. Section VII presents experimental results validating RFL's effectiveness. Finally, Section VIII concludes the paper and identifies potential avenues for future research. Key symbols used throughout the paper are summarized in Table I.

II. Background

This section provides an overview of federated learning for edge computing and emphasizes the importance of data security in the context of new client integration, where newly joining ECs establish keys, upload their first updates, and thus become prime targets of IFGIA.

A. Global Privacy

Global privacy mechanisms are crucial for mitigating the risk of model data leakage across various communication channels and ensuring the confidentiality of edge client identities in federated edge learning [18]. These mechanisms enable collaborative training of a global model by the central server and edge clients without exposing sensitive model or gradient information. The process involves the following steps:

- Security Level Determination: The central server (CS) selects a subset of edge clients for each training round based on factors such as online status and privacy budget utilization.
- Model Data Security during Upload: Edge clients add noise to the gradients before transmission to the CS, adhering to the allocated privacy budget.
- Model Data Security during Broadcast: The CS aggregates the received gradient information from edge clients and assesses the necessity of additional noise injection based on predetermined security levels.

By incorporating noise addition mechanisms at both edge clients and the CS, global privacy measures thwart adversaries attempting to exploit communication channels for data interception. This approach eliminates the need for encryption and decryption of gradients, making it suitable for resource-constrained edge clients. However, the introduction of noise at scale proportional to model dimensions can impact the convergence rate of the global model.

B. Verification of Identity and Integrity

Federated learning for edge computing operates as a decentralized machine learning framework, wherein edge clients train models locally and communicate intermediate results to the CS, responsible for model aggregation [1], [2], [27], [28]. The iterative process involves the following stages:

- Edge Client Selection: The CS dynamically selects a subset of online edge clients and distributes current model parameters. Client inclusion is contingent on real-time availability, with clients subject to removal at any point.
- Local Training: Edge clients independently train their local datasets, iteratively updating the model. To minimize interactions with the CS, each edge client undergoes multiple local training iterations before transmitting updated gradients.
- Model Aggregation: The CS orchestrates the aggregation of intermediate results from edge clients to derive the global model information.
- Model Update: The CS updates the global model and evaluates compliance with predefined criteria, initiating subsequent iterations if necessary.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

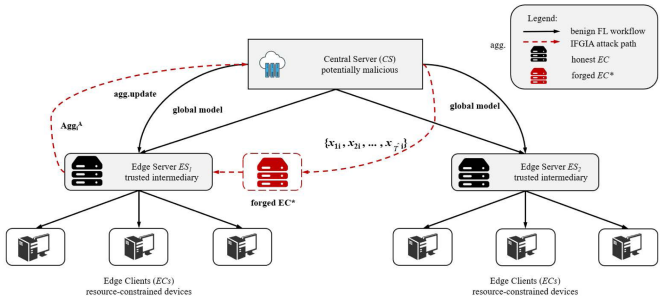


Fig. 1. IFGIA workflow. The central server (CS) resides in the cloud and coordinates multiple trusted edge servers (ESs), each serving a group of resource-constrained edge clients (ECs).

Despite employing secure aggregation techniques to uphold integrity, the CS maintains control over the training process, posing risks of tampering with aggregation outcomes [18]. Moreover, limitations in computing resources and unreliable network conditions expose edge clients to forged identity attacks orchestrated by the CS [29], [30], [31].

III. Identity Forgery and Gradient Inversion Attack

We rigorously elucidate the presence of the identity forgery and gradient inversion attack and launch it on verifiable schemes [29], [30], [31] that are based on secure aggregation (SA). We consider the three-tier CS–ES–EC architecture described in Fig. 1. In this setting, ESs are deployed at the network edge (e.g., base stations or roadside units) and are entrusted to aggregate updates from nearby ECs, while the CS resides in the cloud and coordinates multiple ESs. A malicious or semi-honest CS can exploit this layered structure by forging virtual EC identities and injecting manipulated gradients that are indistinguishable from honest clients from the viewpoint of ESs, thereby deceiving ESs and compromising the privacy of real ECs. We also demonstrate the feasibility of our attack. To facilitate the understanding of the attack process, we present the method of using secret sharing and double aggregation to realize secure aggregation.

A. Secret Sharing

The t-out-of-n secret sharing protocol SS requires each edge client to divide the secret into n pieces and the number of online clients must be no less than $t \leq n$ to reconstruct the secret [32]. When the number of online edge clients is less than t, the secure aggregation cannot be completed. The secret-sharing protocol consists of the following algorithms:

- $Ip \leftarrow SS.Init(1^k)$: This algorithm takes security parameter k as input and outputs an initial parameter Ip.
- $\{p_{u,v}\} \leftarrow SS.Share(t, E, s)$: This algorithm takes threshold t, the set of edge clients E, and the secret s as input, and outputs a set of secret shares $\{p_{u,v}\}_{v\in E_i, u\in E_i, E_i\subseteq E, |E_i|\geq t}$. $p_{u,v}$ is the secret share, which is sent to client v from client u.
- $s \leftarrow SS.Recovery(t, \{p_{u,v}\}_{E_i\subseteq E})$: This algorithm takes threshold t and a set of secret shares $\{p_{u,v}\}$ as input and outputs the secret s.

Algorithm 1: IFGIA: Identity Forgery and Gradient Inversion Attack.

Require: A set of gradients $\{x_1^{Fab}, x_2^{Fab}, \ldots, x_T^{Fab}\}$ of fabricated clients
Require: Aggregation gradients list $\{Agg_1^A, Agg_2^A, \ldots, Agg_T^A\}$ of the targeted client
1: Infer the target gradient as Eq. (3)
2: After T iterations, obtain $\{x_1^{tar}, x_2^{tar}, \ldots, x_T^{tar}\}$
3: **for** $t = 1; t \leq T; t++$ **do**
4: **for** $j = 1; j \leq N; j++$ **do**
5: Calculate reconstructed gradients as Eq. (5)
6: Update: $D_i'^t$ and $Label_i'$ as Eq. (4)
7: **end for**
8: **end for**
9: **return** $\{label'^1, label'^2, \ldots, label'^T\}$.

CS coordinates all gradients uploaded by edge clients, and gradients and the corresponding masks are encrypted and distributed after sharding locally. Therefore, edge clients can verify the correctness of the result according to the value of the aggregated mask and the aggregated gradient value while ensuring the security of the edge client's data by sequentially executing the following processes:

- Edge clients $u \in \mathcal{U}$ obtain the secret slices of other corresponding clients through key sharing and cover up the gradient $\mathbf{y}_u = \mathbf{x}_u + \sum_{v\in\mathcal{U}, u<v} p_{u,v} - \sum_{v\in\mathcal{U}, u>v} p_{v,u}$, where $p_{u,v}$ and $p_{v,u}$ are secret slices and $p_{u,v} - p_{v,u} = 0$.
- Edge clients $u \in \mathcal{U}$ locally calculate the same secret vector pair (a, b), and use (a, b) to encrypt the gradient $\overline{\mathbf{y}}_u = \mathbf{a} \circ \mathbf{x}_u + \mathbf{b}$, where $\circ$ is a Hadamard product.
- CS aggregates $\overline{\mathbf{y}}_u$ and $\mathbf{y}_u$ respectively and obtains $\mathbf{Z} = \sum_{u\in\mathcal{U}} \mathbf{y}_u$ and $\overline{\mathbf{Z}} = \sum_{u\in U} \overline{\mathbf{y}}_u$.
- Edge clients $u \in \mathcal{U}$ can check locally $\overline{\mathbf{z}} \stackrel{?}{=} \mathbf{a} \circ \mathbf{z} + |\mathcal{U}| \cdot \mathbf{b}$, if it does not hold, all clients abort.

During this process, the central server cannot learn the gradient of a specific edge client from the encrypted mask, because the data uploaded by each client is added to the mask of all other client interactions. $VERSA$ [29] uses double aggregation and secret sharing to verify the integrity of aggregation results and guarantee the security of clients' model parameters.

B. The Overview of IFGIA

The identity forgery and gradient inversion attack exploits two features of CS and edge clients on $VERSA$ [29].

1) The number of online edge clients is limited by equipment and network reasons, and there are large deviations. Considering efficiency and communication costs, CS generally only selects a part of clients to participate in training. Therefore, the number of clients participating in training each time is limited.
2) CS controls the training processes in VERSA, and the edge clients that are fabricated by the CS are more likely to be selected. The fabricated edge clients are online during the training and perform the entire aggregation process. Therefore, it is easy for CS to control the number of

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

fabricated edge clients in the verification process to occupy an absolute majority.

These observations lead us to hypothesize that $VERSA$ is vulnerable to IFGIA, and gradients uploaded by clients can be reconstructed. Preventing private data of clients from leakage is the most important security issue to guarantee confidentiality in federated learning [29], [30], [31]. In general, CS can restore the gradient information uploaded by the client by adjusting the number of fabricated clients participating in the client according to SA settings and network fluctuations as shown in Algorithm 1.

We assume that the central server fabricates $\hat{T} \geq Thre$ edge clients and $Thre$ is the threshold of secure aggregation. The central server groups the target edge client and the forged clients together, and separates the other non-target clients into another group, enabling them to execute a secure aggregation protocol separately as

$$Agg^A = x^{tar} + \sum_{i=1}^{i=\hat{T}} x_i, \tag{1}$$

$$Agg^B = \sum x^{non}, \tag{2}$$

where x^{tar} is the gradients of the target client, $\sum_{i=1}^{i=\hat{T}} x_i$ is the aggregation gradients of fabricated clients, and $\sum x^{non}$ is the aggregation gradients of the untarget clients. We set $x^{Fab} = \sum_{i=1}^{i=\hat{T}} x_i$.

The central server trains the global model with updates Agg^B and infers the target gradient as

$$x^{tar} = Agg^A - \sum_{i=1}^{i=\hat{T}} x_i. \tag{3}$$

After T iterations, the central server gets a set of target gradients $\{x_1^{tar}, x_2^{tar}, .., x_T^{tar}\}$. The central server reconstructs the raw datasets and labels according to $x_1^{tar}, x_2^{tar}, .., x_T^{tar}$ as

$$D'_{t+1}, label'_{t+1} = \arg \min_{D'_t, label'_t} ||x_t^{tar} - x'_t||^2, \tag{4}$$

where D'_t and $Label'_t$ are the reconstructed samples and reconstructed labels generated locally by IFGIA at iterations t, x'_t is the gradient by training on reconstructed D'_t, and

$$x'_t = \nabla F(w_t, D'_t), \tag{5}$$

where F is the loss function of model, and w_t is the model weight at the t-th iteration. Finally, IFGIA obtains a set of reconstructed labels $\{label'^1, label'^2, \ldots, label'^T\}$.

C. Experimental Results of IFGIA

We simulate a federated edge computing environment to verify our hypotheses, we run deep neural networks and secret sharing protocols on the standard MNIST dataset[1] and CIFAR-10 dataset[2] on each edge client, respectively. Every edge client sends the encrypted secret slices to CS. By fabricating edge clients, CS obtains all secret fragments and decrypts information submitted by edge clients by IFGIA. To simulate the edge clients, each client has various numbers of samples (1-128 for MNIST and 1-200 for CIFAR-10) and holds samples with different labels.

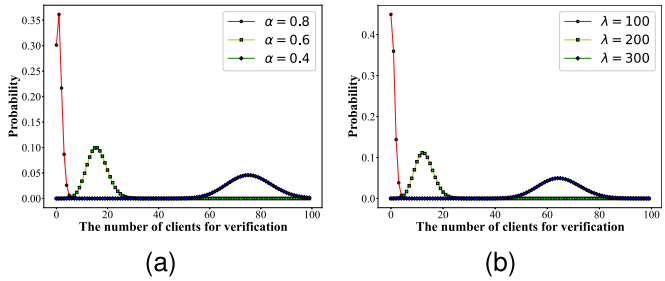


Fig. 2. (a) and (b) compare the probability of different edge clients' numbers for verification with the Uniform distribution and the Poisson distribution, respectively. (a) shows the probability of different numbers with various Uniform distributions; (b) shows the probability of different numbers with various Poisson distributions.

For the secret sharing method, we adopt standard $t - out - of - N$ Shamir secret sharing protocol [32], which is used in [29], [30], [31]. We use SHA-256 and elliptic-curve Diffie-Hellman for pseudo-random generator and key agreement, respectively. We set the threshold value $t = 10$, edge clients $n = 500$, and various dropout ratio α [29]. Because the connection between the edge client and CS is not stable, we assume that the online distribution of the edge clients obeys the Poisson distribution (links to edge clients and edge servers) [33], [34] and uniform distribution [35], [36], respectively. IFGIA is run on one Ubuntu 20.04 LTS server. The SS was conducted on a system equipped with 32 G RAM and an Intel Core i7-8700 processor with a 3.2 GHz base clock.

According to different online distributions, the number of edge clients that can participate in training and verification decreases significantly as the dropout rate increases in federated learning for edge computing. Fig. 2 shows the number of online clients that can participate in verification varies with the dropout ratio in the two distributions. When the number of clients is much lower than the initial number of 10% for verification, CS often only selects a very small number of clients to participate in training. Therefore, the threshold for recovering the secret will be very small. In other words, CS can infer the parameters uploaded by other clients as long as it fabricates a small number of clients, such as 5% or less of the total number.

We set three different thresholds for secret sharing and different numbers of fabricated edge clients. When the system is running, CS preferentially selects fabricated clients to participate in training and verification and constantly checks whether the number $\beta \geq t$ of online fabricated clients is satisfied. If not, CS can not reveal the secret. Because of the unstable connection of edge clients, the probability of a successful attack decreases with the increase of α. Increasing the number of fabricated clients increases the attack success rate of IFGIA. We set different thresholds t and adjust the number of fabricated clients β according to the dropout rate α. In our experiments, we select

[1] http://yann.lecun.com/exdb/mnist/
[2] https://www.cs.utoronto.ca/kriz/cifar.html

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

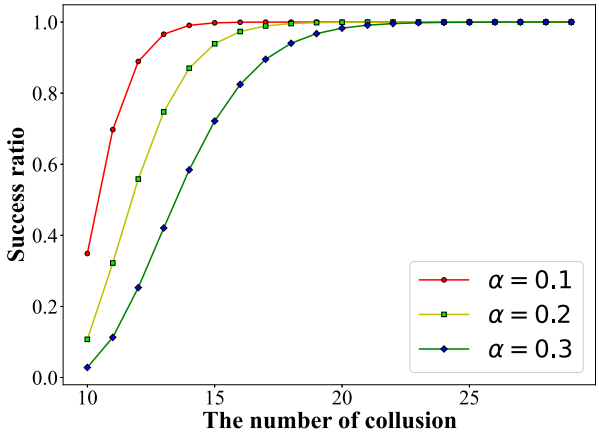


Fig. 3. Success ratio of our attack.

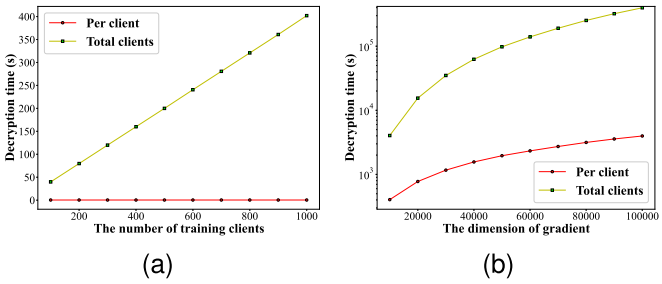


Fig. 4. (a) and (b) compare the computation overhead for decryption with various numbers of training clients and different gradient dimensions, respectively. (a) shows the decryption time with various numbers of training clients; (b) shows the decryption time with various dimensions of gradients.

50 edge clients and launch 100 attacks against each t and β to measure the success rate.

Figs. 3, 4 summarize our 100 attack results on the VERSA [29] model, which contains the success ratio and the time overhead to recover local gradients. Attack results are evaluated in terms of success ratio and efficiency. We analyze the success ratio with the ratio of the number of gradients uploaded by edge clients. We analyze the efficiency by analyzing the time it takes to complete IFGIA and reconstruct local raw datasets.

First, we explain the attack results in terms of success rate. Note that once IFGIA is successful, CS can fully reveal the unmasked gradient information submitted by all edge clients and reconstruct the raw dataset of edge clients. The factors that affect the attack success rate include the number of fabricated edge clients and the dropout rate. When the number of fabricated clients is $\beta = 20$, the attack success rate of our attack all exceed 90% with dropout rates $\alpha = 0.1$, $\alpha = 0.2$, and $\alpha = 0.3$. When we increase β, the attack success rate increases, as shown in Fig. 4. For instance, when the number of fabricated clients reaches 30, the attack success rate under different dropout rates exceeds 99.5%. The results show that increasing the number of fabricated clients can significantly increase the attack success rate. Considering the dropout rate, CS needs to adjust the number of fabricated edge clients according to the dropout situation, and our attack can fully reconstruct the private information under any conditions.

The time required to reveal the gradient is also an important factor affecting the attack results. As shown in figure 4, the time for recovering the raw dataset of target clients is mainly affected by the number of edge clients participating in training and the gradient dimension. Because CS needs to decrypt the gradient of each edge client, taking the AES-OFB mode with a 256-bit key in symmetric encryption as an example, it takes an average of 387 ms for CS to decrypt the gradient for each edge client, when the gradient dimension is 10000. When the dimension increases, the decryption time will grow linearly. For instance, when the dimension is 100000, CS needs 3977 ms to decrypt the gradient. However, the time for the revealing gradient is still acceptable. Therefore, the results of the attacks prove that IFGIA in edge computing can fully reconstruct all local gradients.

D. Analysis of IFGIA

In this subsection, we summarize the security implications of IFGIA and clarify its scope, assumptions, and limitations, instead of re-stating general background or survey-style material.

IFGIA relies on the following key preconditions: (i) the central server (CS) is semi-honest or malicious and is allowed to select participating edge clients in each round, (ii) the CS can fabricate arbitrary virtual client identities that are indistinguishable from honest clients from the viewpoint of ESs and the secure aggregation (SA) protocol, and (iii) the SA protocol exposes to the CS the aggregate of local gradients over the clients selected in a round. Under these conditions, if the CS can ensure that a target client and a sufficient number of fabricated clients are grouped together, it can algebraically isolate the target gradient via (1)–(3), and then run gradient inversion on the recovered gradient sequence as in (4)–(5).

The above preconditions are naturally met by a family of SA-based verifiable FL schemes such as VERSA and its variants [29], [30], [31]. These schemes are designed to hide individual gradients from a semi-honest CS while allowing the CS to obtain the sum of gradients and, in some cases, homomorphic hash values for verifiability. However, once the CS can populate the SA groups with a majority of fabricated clients (e.g., by exploiting high and fluctuating dropout rates in edge environments), the SA threshold condition becomes an advantage for the adversary: the CS can simply treat fabricated gradients as known variables and solve for the unknown target gradient.

Our experimental results in Section III-C quantify this effect under realistic client dropout distributions, showing that when the number of fabricated clients reaches a fraction of the online population (e.g., $\beta \geq 20$ out of $n = 50$ selected clients), the CS can recover unmasked gradients with success rates exceeding 99.5% for typical parameter settings.

Despite its high success probability under favorable conditions, IFGIA is not unconstrained:

- It targets gradient-based models such as deep neural networks; it does not directly apply to models trained without gradients (e.g., decision trees).

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

- The attack requires a sufficient number of fabricated clients to satisfy the SA threshold and to dominate the group containing the target client; when the fraction of fabricated clients is small or the threshold is high, the success probability decreases.
- Gradient inversion itself becomes harder when additional randomness (e.g., differential privacy noise, dropout, or data augmentation) is applied to the training process; in such cases, the reconstructed samples and labels suffer from degraded fidelity.

Prior work has shown that gradients can leak significant information about local training data in federated learning [30] and that even SA-based protocols may be vulnerable to malicious servers that deviate from the protocol [29]. IFGIA inherits the basic idea of reversing gradients but systematically exploits the CS's ability to fabricate clients and to control group formation in edge settings with high dropout rates. Our contribution in this section is to explicitly cast these behaviors as a concrete attack, quantify its success probability under different client distributions and thresholds, and use this as a motivating threat model for the design of the RFL framework in Section V.

IV. System and Threat Models

This section delineates the system and threat models inherent to the RFL system.

A. System Model

The federated edge learning system consists of N edge clients (ECs), M edge servers (ESs), and a central server (CS).

- Central Server (CS): Operating akin to cloud service providers, the CS initiates and oversees federated learning tasks. It aggregates model parameters from ESs and divides the global model into shallow and deep layers, depending on EC capabilities. The CS extracts partial training sample information and allocates ES computing resources, balancing the varying online presence of ECs.
- Edge Server (ES): High-performance devices like UAVs and small base stations, ESs gather and aggregate local ECs' model parameters, train the deep layer model, and validate aggregation results.
- Edge Clients (ECs): These include diverse smart devices like vehicles and phones, responsible for training local shallow layer models and sending updates to their nearest ES. ECs, CS, and ESs interact using encrypted data. ESs and CS, driven by interest, provide services to ECs while seeking access to their private data.

B. Threat Model

We detail the threat model, which has a central server, m edge servers, and n edge clients. The central server controls the training process and is interested in the edge clients' private information. Edge clients and edge servers train the model parameters in good faith. The count of online edge clients is restrained due to hardware and network constraints, resulting in significant variances. Simultaneously, the CS manipulates the training processes by forging a fraction of the edge clients. Fabricated edge clients remain online during the training and execute the entire aggregation procedure. Consequently, it facilitates the CS's control over the aggregation process.

We posit that the CS is semi-honest or malicious respectively [37]. When CS is semi-honest, CS fairly coordinates each edge client's training process but may fabricate edge clients to infer private information by reconstructing the raw datasets of edge clients. When CS is malicious, CS further deceives edge clients into accepting a counterfeit global model. Specifically, CS controls the attack aims by dominating the dummy gradients x^{Fab}. We postulate that the maximum quantity of edge clients that the CS can corrupt is ζ, where $0 \leq \zeta \leq n-1$ and n represent the total number of edge clients. In particular, the CS not only orchestrates the training process but also has the ability to forge virtual EC identities and inject manipulated gradients, with the goal of deceiving ESs and extracting private information from honest ECs.

In addition to fabricated edge clients, we also exclude poisoning attacks, where malicious clients submit deceptive or tailored data to misguide the training process and degrade model performance. Although poisoning attacks are a valid concern, they are not within the scope of this work, as our primary focus is on protecting privacy and model integrity from the perspective of client-level attacks. Future work could explore countermeasures against poisoning attacks, potentially by using advanced techniques such as non-interactive zero-knowledge proofs, though these methods come with computational and communication costs that may not be feasible for resource-constrained edge clients.

V. RFL Framework

To defend against IFGIA, we propose robust federated learning for edge computing. RFL aims to enhance the privacy and verify the identities of edge clients for local edge clients. This section describes the technical overview of RFL and details the design and implementation of RFL.

A. Cryptographic Primitives

1) Digital Signature: Digital signatures (DS) use public key infrastructure to establish a verifiable mechanism that links edge clients with the messages they send. The following algorithms summarize DS:

- $(sk, pk) \leftarrow DS.Ini(\lambda)$: This algorithm accepts a security parameter λ as input and generates a pair of signing and public keys (sk, pk) [30].
- $z \leftarrow DS.Sign(sk, x)$: This algorithm takes the secret key sk and model parameters x as input, and outputs a signature z.
- $\{0, 1\} \leftarrow DS.Verify(pk, x, z)$: This algorithm takes the public key pk, model parameters x, and signature z as input, and returns a verification result of 0 or 1 that indicates failure or success, respectively.

2) Homomorphic Hash: Homomorphic hashing (HH) allows edge clients to locally compute the hash values of other edge clients to verify the integrity of aggregation results, which

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: RFL: Robust Federated Learning Framework for Edge Computing.

Require: All edge clients u generate key pair (sk_u, pk_u).
Require: All ESs v generate key pair (sk_v, pk_v).
Require: CS initializes the global model and broadcasts $\{pp, p^{(0)}\}$ to all ESs.
Require: $p_u^{(0)} = p^{(0)}$
1: Each u sends pk_u to ESs.
2: Each v sends pk_v to other ESs.
3: CS trains and splits the global model into the shallow layers model and deep model.
4: **for** $t = 1; t \leq T; t++$ **do**
5: **for** Every edge client u **do**
6: u locally trains the shallow layers model as Eq. (6)
7: Perturb the gradients as Eq. (7)
8: Sign the gradients as Eq. (8)
9: Send the gradient and the signature to ES
10: **end for**
11: **for** Every edge server v **do**
12: ES verifies the signatures as Eq. (9)
13: Process forward propagation process as Eq. (10)
14: ESs and edge clients collaboratively process the negative feedback propagation
15: Compute h_v of p_v as Eq. (13)
16: Sign h_v as Eq. (14)
17: Advertise (h_v, z_v).
18: **end for**
19: CS coordinates gradients as Eq. (15)
20: CS broadcasts gp
21: **for** Every edge server v **do**
22: Verify z_v as Eq. (16)
23: Verify gp as Eq. (17)
24: If it does not hold, v exits.
25: **end for**
26: **end for**
27: **return** gp.

were broadcast by the cloud server. The following algorithms summarize the HH:

- $pp \leftarrow HH.HGen(1^k, 1^d)$: This algorithm takes the security parameter k and the dimension d as input, and generates the public parameter pp [31].
- $h \leftarrow HH.Hash(x)$: This algorithm accepts a d-dimensional vector x as input and outputs the homomorphic hash of $h \leftarrow \prod_{i\in[d]} g_i^{x[i]} \bmod o$.
- $h^* \leftarrow HH.Eval(h_1, \ldots, h_l)$: This algorithm accepts l hashes and l coefficients as input and computes the aggregation $h^* = \prod_{i\in[l]} h_i \bmod o$.

B. Proposed Scheme RFL

The main goal of RFL is to prevent private data from leakage during the whole training process and verify the correctness of the results and the identity of edge clients to achieve lightweight and efficient performance on edge clients. The details of RFL are provided in the Algorithm 2.

Every edge client u and edge server v participating in training locally generates key pairs (sk_u, pk_u), (sk_v, pk_v) for verification, respectively. Edge client u sends public keys $\{pk_u\}$ to ES v, and ESs exchange public keys $\{pk_v\}$ and $\{pk_u\}$ directly.

Then, the CS initializes the global model and logically splits it into a shallow part and a deep part according to the hardware resources of the edge clients. We denote the parameters of the shallow part by p_α and those of the deep part by p_β. The CS broadcasts p_α to all edge clients and p_β to all edge servers, and distributes the public parameters pp of the homomorphic hash to all ESs.

Edge client u trains the shallow layer locally as

$$p_u = SL(p_\alpha, D_u), \tag{6}$$

where SL is the shallow layer and SL extracts the features of local dataset D_u and outputs p_u. Then edge client u perturbs the intermediate parameters with additional noise $\eta_u \sim N(0, \sigma_u^2)$ to guarantee the data security as

$$\hat{p}_u = p_u + \eta_u, \tag{7}$$

where $\sigma_u = \frac{cd_\alpha \Delta_2^S}{\epsilon}$, ϵ is the privacy budget, d_α is the dimension of p_u, and Δ_2^S is the sensitivity. According to different privacy levels, u adds different scales of noise. Specifically, we can calculate the sensitivity based on the model dimension and gradient fluctuation range as

$$\begin{aligned}\Delta_2^S &= \max_{D_u, D_u'} ||p_u' - p_u|| \\ &= \max_{D_u, D_u'} \left|\left| \frac{1}{|D_u|} \sum SL(p_\alpha, D_u) \right|\right| \\ &\quad - \frac{1}{|D_u'|} \sum SL(p_\alpha, D_u')|| \\ &= \frac{2C}{|D_u|},\end{aligned}$$

where C indicates the clipping threshold, D_u' denotes the adjacent dataset of D_u. Edge client u locally signs the $\hat{p}_u$ as

$$z_u \leftarrow DS.Sign(sk_u, \hat{p}_u). \tag{8}$$

Then, u sends $(z_u, \hat{p}_u)$ to the nearest v.

After receiving $(z_u, \hat{p}_u)$, the edge server v checks signature as

$$\{0, 1\} = DS.Verify(pk_u, \hat{p}_u, z_u), \tag{9}$$

where $DS.Verify$ is a signature algorithm, inputs pk_u and z_u, and outputs 0 or 1. Then v trains subsequent deep layers models based on the received intermediate data as

$$p_v = DL(\hat{p}_u, p_\beta), \tag{10}$$

where DL is the deep layer and DL processes forward propagation process of a neural network and outputs p_v. Then, v sends p_v to edge client u.

After receiving p_v, the edge client u computes the gradient as

$$g_u = \nabla Loss(p_v, D_u), \tag{11}$$

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

where $Loss$ is the loss function. Then u sends g_u to v, and v uses the returned gradient information g_u in the deep network part to update its gradient as

$$g_v = Update(g_u), \tag{12}$$

where $Update$ is the next negative feedback propagation and its output is g_v.

To verify the integrity of the aggregation results, ES v needs to calculate the hash value of the gradient as

$$h_v \leftarrow HH.Hash(g_v), \tag{13}$$

and sign the hash value as

$$z_v \leftarrow DS.Sign(sk_v, h_v), \tag{14}$$

and then sends g_v to CS and broadcasts h_v, z_v to other ESs. CS aggregates all gradients as

$$gp = p_\beta + \frac{1}{M}\sum g_v, \tag{15}$$

where M is the number of edge servers. Then CS broadcasts gp to all ESs.

After receiving gp and verification data, ES v checks signature of all other ESs $\{(h_i, z_i)\}_{i\neq v}$ as

$$\{0,1\} = DS.Verify(pk_i, h_i, z_i), \tag{16}$$

where $DS.Verify$ is a signature algorithm, inputs pk_i, h_i, and z_v and outputs 0 or 1. If $DS.Verify$ returns 0, and v exits from the system. Otherwise, v verifies $\sum_{v\in\mathcal{M}} g_v$

$$\prod_{v=1}^{M} h_v \equiv HH.Hash\left(M\cdot(gp - p_\beta)\right) \pmod{o} \tag{17}$$

If (17) does not hold, all ESs break away from RFL.

According to the post-processing properties of DP, the private information of the interaction gradients is preserved during the uploading channels and broadcasting channels. In the meantime, ES v can locally compute the hash value of aggregation and verify the correctness of the results.

VI. Analysis of Proposed RFL

This section analyzes the security and efficiency of RFL.

A. Security of RFL

This section focuses on the protection of private data and the verification of RFL. Theorem 1, Theorem 2, and Theorem 3 show that our proposed RFL achieves global privacy and verification of the identity of edge clients and integrity of aggregation.

Theorem 1: RFL satisfies (ϵ, δ)-DP during the updating phase and broadcasting phase for edge clients.

Proof: The edge client u adds Gaussian noise to the intermediate parameters. The perturbed parameter is given by:

$$\hat{p}_u = p_u + \eta_u,$$

where $\eta_u \sim \mathcal{N}(0, \sigma_u^2)$.

The scale of the noise is defined as:

$$\sigma_u = \frac{cd_\alpha \Delta_2^S}{\epsilon},$$

where ϵ is the privacy budget, d_α is the dimension of p_u, and Δ_2^S is the sensitivity. The sensitivity is calculated as:

$$\Delta_2^S = \max_{D_u, D_u'} \|p_u' - p_u\| = \frac{2C}{|D_u|},$$

where C indicates the clipping threshold, and D_u' denotes the adjacent dataset of D_u.

In RFL, edge client u adds Gaussian noise $\eta_u \sim \mathcal{N}(0, \sigma_u^2)$ to the intermediate parameters p_u, and

$$\sigma_u = \frac{cd_\alpha \Delta_2^S}{\epsilon} = \frac{2cd_\alpha C}{|D_u|\epsilon},$$

where $c = \sqrt{2\ln(1.25/\delta)}$ and δ is the standard parameter in the Gaussian mechanism.

For any two adjacent datasets D and D', and for any set of outputs O, the Gaussian mechanism $\mathcal{M}$ satisfies:

$$\Pr[\mathcal{M}(D) \in O] \leq e^\epsilon \Pr[\mathcal{M}(D') \in O] + \delta.$$

Here, $\mathcal{M}$ represents the process of adding Gaussian noise to p_u.

Therefore, each upload from an edge client satisfies (ϵ, δ)-differential privacy for the uploading channel. According to the post-processing property of differential privacy [16], any subsequent computation and communication over CS and ESs based on these noisy parameters also preserves (ϵ, δ)-DP. Thus, RFL satisfies (ϵ, δ)-DP during both the updating and broadcasting phases for edge clients. □

In the following, we analyze the verification of aggregation gp and the identity u.

Theorem 2: In the context of RFL, each edge server possesses the capability to autonomously verify the identities of participating edge clients u through a combination of cryptographic and algorithmic procedures.

Proof: In the framework of RFL, the verification process of the identity of an edge client u by an edge server v is initiated by both entities generating distinct key pairs, denoted as (sk_u, pk_u) and (sk_v, pk_v), respectively.

Subsequently, the edge clients transmit their corresponding public keys $\{pk_u\}$ to their designated edge server v.

The integrity and non-repudiation of the data sent from the edge client to the edge server are ensured by the generation of a signature z_u for the perturbed parameter $\hat{p}_u$, achieved through the utilization of the client's private key as depicted below:

$$z_u \leftarrow \text{DS.Sign}(sk_u, \hat{p}_u)$$

This operation plays a critical role in guaranteeing the authenticity of the data's source and its immunity to tampering.

Subsequently, edge server v conducts the verification of the aforementioned signature, making use of the following operation:

$$0, 1 \leftarrow \text{DS.Verify}(pk_u, h_i, z_u)$$

The DS.Verify operation is of paramount importance in ascertaining the legitimacy of the data source and validating its integrity, as it confirms that the signature z_u was indeed generated with the private key associated with the public key pk_u.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

In conclusion, this procedure exemplifies the robustness of the identity verification process within the RFL framework, establishing trust in the communication and data transfer between edge clients and their respective edge servers. ■

Theorem 3: In the context of RFL, each edge server possesses the capability to autonomously verify the correctness of the aggregated model parameters gp.

Proof: Let us consider an edge server v within the RFL system, which initiates the process of verifying the correctness of the aggregated model parameters. To commence this verification, edge server v generates a key pair (sk_v, pk_v) and exchanges these public keys with other edge servers (ESs) in the system.

Subsequently, edge server v undergoes the training process for the deep layers of the model, denoted as DL, with the utilization of the following equation:

$$p_v = DL(\hat{p}_u, p_\beta)$$

Here, DL represents the deep layer model, and p_v is the result of this training.

Concurrently, the edge client u computes the gradient g_u as:

$$g_u = \nabla Loss(p_v, D_u)$$

Where $Loss$ signifies the loss function, and D_u denotes the dataset associated with edge client u.

Subsequently, edge server v updates its gradient g_v through a process referred to as $Update(g_u)$.

To enhance the security of the verification process, edge server v computes the hash of its gradient, given by:

$$h_v \leftarrow HH.Hash(g_v)$$

Following this, edge server v signs this hash h_v using its private key, resulting in the signature z_v:

$$z_v \leftarrow DS.Sign(sk_v, h_v)$$

The Central Server, responsible for aggregating gradients, computes the aggregated gradient gp as follows:

$$gp = p_\beta + \frac{1}{M}\sum g_v$$

Where p_β represents the global model parameters, and M denotes the total number of edge servers.

Upon receiving the aggregated model parameters gp, each edge server v initiates the verification of signatures from all other participating edge servers. This verification is carried out through the operation:

$$0, 1 \leftarrow DS.Verify(pk_i, h_i, z_i)$$

Where pk_i, h_i, and z_i correspond to the public key, gradient hash, and signature of each participating edge server i.

Lastly, each edge server v verifies the aggregated model gp by confirming that the product of all h_v values modulo o is congruent to the hash of gp computed through:

$$\prod_{v=1}^{M} h_v \equiv HH.Hash\left(M \cdot (gp - p_\beta)\right) \pmod{o}$$

In conclusion, this rigorous verification process underscores the reliability and correctness of the aggregated model parameters in the RFL system, ensuring the integrity of federated learning operations. ■

B. Efficiency of RFL

This section analyzes the efficiency in terms of the convergence, communication overhead, and computation overhead of RFL.

In RFL, the CS splits the global model into a shallow part and a deep part. The shallow part is deployed on edge clients, while the deep part is deployed on edge servers. In our implementation, we fix the architecture of the shallow part to have a constant dimension and place only a small number of layers on each client. The remaining layers are trained on ESs. By reusing the locally extracted features across rounds and avoiding full-model training on edge clients, RFL reduces the per-client computation overhead and accelerates convergence compared with schemes that require clients to train the entire global model.

Computational overhead and communication overhead are linearly related to model dimensions, which can lead to challenges when deploying machine learning models on resource-constrained edge clients. To address these challenges, edge clients train a shallow layers model with a fixed dimension length in RFL.

Theorem 4: In the RFL framework, both the communication and computation overheads are invariant with respect to the dimensionality (L) of the global model.

Proof: Consider RFL where the global model is partitioned into a shallow layer and a deep layer. Denote the dimension of the global model by $d_G = L$, with d_α and d_β representing the dimensions of the shallow and deep layers respectively. Thus, we have:

$$d_G = d_\alpha + d_\beta.$$

The shallow layer is deployed on edge clients. Assume that the computation overhead at the edge clients is a function of the dimension of the shallow layer (d_α) and the number of training samples (TS). This overhead can be expressed as:

$$Cost_A = O(TS \cdot d_\alpha).$$

Given that d_α is a constant (i.e., $d_\alpha = O(1)$) and does not scale with d_G, the computation overhead simplifies to:

$$Cost_A = O(TS).$$

This shows that the computation overhead at edge clients is independent of the dimension of the global model.

For the communication overhead, consider the data transmitted between the edge clients and the central server. Since the deep layer (d_β) is not involved in this communication and d_α is constant, the communication overhead also remains constant regardless of d_G.

Hence, both computation and communication overheads in the RFL framework are invariant with respect to the global model's dimension L. ■

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DATASETS

Dataset	MNIST	CIFAR-10
Type	Gray	Color
Training Images	60000	50000
Test Images	10000	10000
Training Images Per Client	128	200
Test Images Per Client	100	100

TABLE III
MODEL NETWORK

Model	MLP Network
Per Hidden Layer	256 hidden units
Feed-forward Neural Network	ReLU & Softmax
Learning Rate	0.002
Network Depth	3-18

TABLE IV
EQUIPMENT ENVIRONMENT

Equipment	Desktop
Operation System	64-bit Ubuntu 16.09 LTS
CPU	Intel i7-8700 CPU (3.20 GHz)
RAM	32 GB

TABLE V
ENCRYPTION SETTINGS

Encryption Algorithm	
Homomorphic Hash	NIST P-256 curve
Key Agreement	NIST P-256 curve with SHA-384
Digital Signature	NIST P-256 curve

C. Limitation of RFL

While the RFL framework provides strong privacy protection and efficient communication, several challenges remain for its practical deployment:

- Computational Load on Edge Servers: Offloading heavy computations to edge servers increases their workload. In resource-constrained edge environments, ensuring sufficient computational resources for deep layer training on edge servers may impact performance.
- Dynamic Client Participation: The fluctuating availability of edge clients due to network instability or device mobility could affect model consistency and accuracy.
- Advanced Privacy Threats: Although RFL reduces privacy risks through model partitioning and gradient perturbation, it is not immune to advanced side-channel attacks or model inversion.
- Communication Efficiency: As the number of clients or model complexity increases, communication overhead could still pose a challenge, especially in high-latency networks.
- Model Partitioning Complexity: Determining the optimal partitioning of the model into shallow and deep layers can be complex, especially for large-scale models or task-specific applications.

The RFL framework offers promising advantages but faces challenges related to edge server resources, dynamic client participation, privacy protection, and communication efficiency. Addressing these through future improvements such as distributed systems, advanced cryptographic techniques, and adaptive strategies will enhance its effectiveness and make it suitable for deployment in real-world federated edge computing environments.

VII. EVALUATION

This section evaluates RFL compared to VERSA [29], VERIFL [30], VerifyNet [31], and NbAFL [18], which are the most relevant state-of-the-art schemes for privacy-preserving and verifiable FL. We conduct a series of experiments on two real-world datasets to demonstrate the convergence, communication overhead, and computation overhead. We evaluate our proposed RFL on two real datasets, MNIST and CIFAR-10. The details of the implementation setup are shown in Table II, Table III, Table IV, and Table V.

In all experiments, the global model is an L-layer MLP as specified in Table III. We deploy the first L_s hidden layers (with $L_s = 1$ by default) as the shallow part on edge clients and the remaining $L - L_s$ layers as the deep part on edge servers. All layers are randomly initialized and jointly trained; no external pre-training is used.

A. Evaluation for Privacy and Convergence

Privacy in the context of federated learning refers to the protection of client data from being exposed through model updates. One of the main privacy risks in federated learning is the ability of an adversary to infer private information about the training data by reconstructing gradients or exploiting side-channel information. We measure privacy primarily by evaluating the success rate of the IFGIA attack. IFGIA aims to extract the unmasked gradients of target clients from aggregated gradients, which could potentially leak private information.

The success rate of IFGIA is used as an indicator of privacy leakage. A lower attack success rate implies better privacy protection. Specifically, RFL is compared to other schemes such as VERSA, VERIFL, VerifyNet, and NbAFL in terms of how well they prevent the extraction of private gradients. The attack success rates are summarized as follows: VERSA, VERIFL, and VerifyNet have attack success rates exceeding 99.5%, meaning the central server can almost fully reconstruct the gradients from these schemes. NbAFL reduces the attack success rate to 91.7% by adding noise to perturb the gradients. RFL, through model partitioning and gradient perturbation, reduces the attack success rate to 73.6%, showing significant privacy improvement over the other schemes. Thus, RFL provides a higher level of privacy protection by reducing the relationship between the model's outputs and the labels, which makes it more resistant to IFGIA.

Convergence refers to how quickly the model parameters reach an optimal or near-optimal value during training. Convergence is typically measured by evaluating the loss function value and accuracy over the course of the training process. The loss function measures the discrepancy between the model's predicted outputs and the actual target labels. A decreasing loss function indicates that the model is improving in its predictions

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

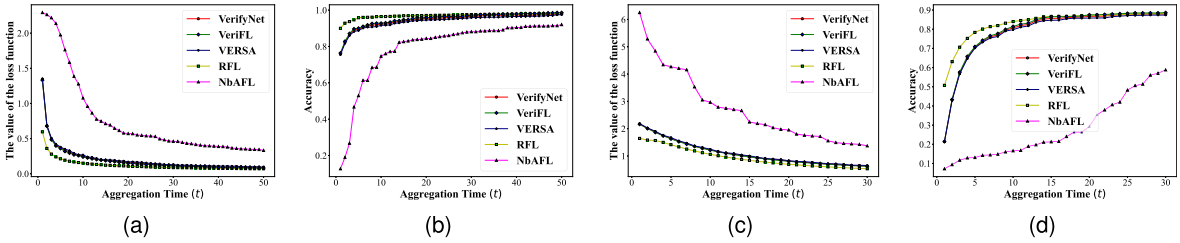


Fig. 5. (a) and (b) show the value of the loss function and the accuracy of the five schemes with various t on the dataset MNIST; (c) and (d) show the value of the loss function and the accuracy of the five schemes with various t on the dataset CIFAR-10, respectively.

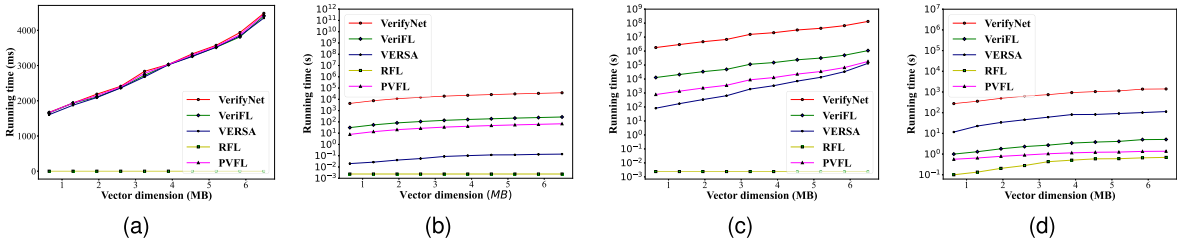


Fig. 6. (a), (b), and (c) show model training time, data encryption time, and total running time of the edge client, respectively. (d) shows the total running time of CS.

over time. Accuracy is calculated as the percentage of correct predictions made by the model on the test set. A higher accuracy indicates that the model has learned the correct patterns and is able to generalize well.

In our experiments, we evaluate the loss function and accuracy for various schemes on both the MNIST and CIFAR-10 datasets. Fig. 5(a) and (b) show the loss function values and accuracy rates for the different schemes during training. RFL demonstrates faster convergence than the other schemes. In contrast, schemes like NbAFL (which adds noise) struggle with convergence due to the added noise, resulting in a slower reduction in the loss function. VERSA, VERIFL, and VerifyNet achieve similar performance in terms of accuracy, as they do not add noise to perturb gradients. These schemes converge at a standard rate, with a smooth reduction in loss over time. NbAFL introduces noise, which slows down the convergence process. As the model's dimension increases, the noise scale increases, which significantly hampers convergence, as shown by the slower decrease in loss.

As shown in Fig. 5(a) and (b), the RFL framework achieves faster convergence, evidenced by the faster decrease in loss and the higher accuracy compared to other schemes. NbAFL shows a slower convergence rate due to its noise addition, while VERSA, VERIFL, and VerifyNet show steady and comparable convergence performance, as no noise is added. RFL not only provides significant privacy protection by reducing the IFGIA attack success rate to 73.6%, but it also achieves faster convergence compared to other privacy-preserving federated learning schemes. The shallow layers in RFL help speed up the learning process, ensuring that the model reaches an optimal state more efficiently than other schemes, which often suffer from slow convergence due to added noise or lack of model partitioning.

B. Performance Evaluation for Computation Overhead

We demonstrate the computation time for the five schemes with various dimensions of the global model on the dataset MNIST as shown in Fig. 6, respectively. VERIFL [30], VerifyNet [31], PVFL [38], and VERSA [29] all require edge clients to train and verify the global model locally. Therefore, as the dimension increases, the model training time, data encryption time, and total running time also increase for both the edge clients and CS. VERSA [29] uses the double mask to protect the private information and verify the aggregations, VERIFL [30] uses homomorphic hashing to verify the correctness of aggregation results, and VerifyNet [31] uses zero-knowledge proofs to verify aggregation results. Meanwhile, PVFL [38] applies DP noise to reduce the communication overhead and computation overhead by single aggregation for verification. Thus, PVFL [38] achieves less computing time and total time than VERIFL. The computation overhead of VerifyNet [31] is greater than VERIFL [30], and the computation overhead of VERIFL [30] is greater than VERSA [29]. RFL proposes the adaptive model splitting mechanism and each client locally trains a fixed number of model dimensions. Therefore RFL achieves dimension-independent computation overhead per client and shorter computing time than VERIFL [30], VerifyNet [31], and VERSA [29]. Thus, the proposed RFL outperforms the other schemes in terms of computation overhead.

C. Performance Evaluation for Communication Overhead

We demonstrate the value of the communication overhead for the five schemes with various dimensions of the global model on the dataset MNIST as shown in Fig. 7, respectively. VERIFL [30], VerifyNet [31], and VERSA [29] all require the edge clients to send the global model to CS. Therefore, their communication overhead is increasing with the dimension for both the edge clients and CS. VerifyNet [31] applies zero-knowledge proofs to verify aggregation results; VERIFL [30] applies homomorphic hashing to verify the correctness of aggregation results; VERSA [29] applies double masking to prevent

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

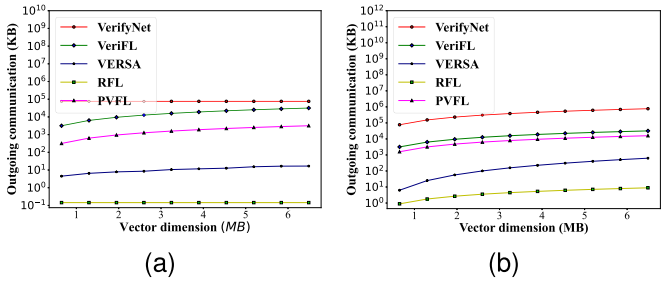


Fig. 7. (a) shows outgoing communication with edge clients; (b) shows outgoing communication of edge CS, respectively.

private data from leakage, respectively. Thus, the communication overhead of VerifyNet [31] is greater than VERIFL [30], and the communication overhead of VERIFL [30] is greater than VERSA [29] and PVFL [38]. In RFL, each client locally trains a fixed number of model dimensions, thus its communication overhead is independent of the dimension of the global model. Therefore, RFL achieves smaller communication overhead than NbAFL [18], VERIFL [30], VerifyNet [31], PVFL [38], and VERSA [29]. Above all, the proposed RFL outperforms the other schemes.

D. Performance Under Different Conditions

In this section, we extend the evaluation of the RFL framework to a broader set of use cases and network conditions, demonstrating its versatility and scalability across various edge computing environments.

Federated learning systems are often deployed in dynamic network environments, where client availability, network instability, and communication bandwidth can fluctuate. We consider the following scenarios: In edge computing, network connections are often unstable, and the dropout rate (the proportion of clients that fail to participate in a training round) can vary significantly. We evaluate RFL's performance when dropout rates range from 5% to 50%, mimicking real-world scenarios where clients may experience intermittent connectivity or fail to join the training process. We observe that RFL maintains robust convergence even with high dropout rates, thanks to its partitioned model architecture and efficient aggregation techniques. The framework can still achieve fast convergence and strong privacy protection without requiring all clients to be present in every training round. In environments with limited bandwidth, the overhead of transmitting model updates can be a significant bottleneck. We evaluate RFL under different network bandwidth conditions, ranging from low to high bandwidth. RFL demonstrates excellent performance under low-bandwidth conditions by reducing the amount of data transmitted between edge clients and central servers. The partitioned model ensures that only partial model updates (the shallow layers) are sent, greatly reducing the communication overhead and allowing edge clients to operate efficiently even with limited bandwidth. The RFL framework is designed to perform efficiently under a variety of network conditions and with different types of edge devices, making it adaptable to a wide range of edge computing environments. Whether dealing with high dropout rates, low bandwidth, or resource-constrained devices, RFL maintains robust privacy protection, fast convergence, and scalable performance. These evaluations provide strong evidence of the framework's flexibility and applicability in real-world federated edge learning scenarios.

VIII. Related Work

This section delineates previous strategies for secure Federated Learning (FL), focusing on preventing data leakage and verifying aggregations.

A. Federated Learning

Traditional centralized machine learning models primarily depend on trusted third parties to amass data and train models, providing services. However, the data exchange between the local smart device and the third-party cloud instigates severe response delays and drastically impairs the service experience. Edge computing, a distributed architecture for data processing at the network's edge, furnishes more efficient computing services via prompt communication responses. Internet of Things (IoT) devices connect to proximate edge servers, utilizing the servers' computational capacity to achieve more reliable communication and computational efficiency. Despite these advances, deploying large-scale model parameters, which can reach trillions, into IoT devices remains a significant challenge. Furthermore, the data of edge clients can be manipulated and substituted during transmission [39], making edge clients highly susceptible to various attacks [15], [18], [19], [38], [40], [41], [42], [43], [44]. To safeguard data privacy in edge computing, federated learning mandates IoT devices to locally train models and share only encrypted intermediate results with edge servers, rather than raw data. This process, dubbed federated learning for edge computing [19], [38], [45], [46], preserves data privacy interacted between IoT devices and edge servers. Federated learning for edge computing achieves model training tasks at the edge, enhances communication efficiency, and maintains data privacy. Nevertheless, local encryption and transmission of large models' global parameters remain a massive challenge for resource-constrained edge clients. Therefore, proposing a novel robust federated edge learning framework to ensure security, effectiveness, and efficiency is imperative.

B. Verifiable Federated Learning

In federated learning, a central server might manipulate the model aggregation results or substitute a fraction of edge clients to infer honest edge clients' private information, thus yielding incorrect classification results [47]. Therefore, verifying the aggregation results' integrity and the edge clients' identity is essential.

Xu et al. [31] utilized homomorphic hashing to verify the integrity of server aggregation results in federated learning. Nonetheless, computation overhead and communication overhead linearly increase with the model dimension. Guo et al. [30]

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

suggest VeriFL verify the model aggregation results' integrity, using linear homomorphic hashing to verify aggregation results, thereby reducing verification's communication overhead. Zhou et al. [19] introduce PFLF that applies digital signatures to verify participant identities and homomorphic hashing to verify aggregation results. Zhou et al. [38] suggest PVFL, combining differential privacy and digital signature to protect privacy and verify integrity while achieving dimension-independent computation overhead for verification. However, [19], [30], [31], [38] all necessitate edge clients to train and verify the global model locally and transmit global model parameters, which imposes substantial communication and computation overhead for edge clients. Hence, these schemes are challenging to apply to resource-constrained edge clients. Amrita et al. [44] suggested EIFFeL to verify aggregation results and local updates' correctness from clients. Yet, EIFFeL requires over two-thirds of the edge clients to continuously and honestly verify aggregation integrity, which is unrealistic for unstable edge clients. Hahn et al. [29] demonstrated that if a Central Server fabricates edge clients, CS can recover original gradients based on the homomorphic hash values submitted by honest edge clients. However, Hahn et al. [29] required more than half of the edge clients to continuously train and verify. The above schemes do not consider verifying the identity of edge clients and the highly related dataset of edge clients, therefore they are vulnerable to IFGIA. To defend against IFGIA, our proposed RFL ensures the correctness of aggregation verification without requiring a threshold of trusted edge clients. Furthermore, RFL can verify the identities of edge clients and reduce the communication overhead and computation overhead for edge clients.

C. Federated Edge Computing

Federated learning has been widely adopted in edge computing scenarios to enable privacy-preserving model training on resource-constrained IoT devices and mobile clients [19], [38], [45], [46]. Compared with traditional cloud-centric training, federated edge learning must cope with three key challenges: (i) heterogeneous and limited computation and storage capabilities at edge clients, (ii) unstable connectivity and high dropout rates due to mobility and wireless links, and (iii) stricter privacy requirements because data are often generated in sensitive domains such as healthcare, finance, and smart transportation.

Existing federated edge learning frameworks mainly focus on reducing communication and computation overhead (e.g., via model compression, client selection, or device-aware scheduling), or on providing basic data privacy against external adversaries by keeping raw data local. However, they typically assume a benign or at most semi-honest central server, and do not jointly address (1) privacy leakage from gradient-based attacks mounted by the server itself, and (2) verifiability of aggregation results and client identities. Moreover, many schemes implicitly require edge clients to train and verify full-dimensional global models, which is not well suited for highly resource-constrained devices.

In contrast, RFL is explicitly designed for federated edge learning with a potentially malicious CS. By combining model partitioning between ECs and ESs, lightweight differentially private perturbation on intermediate representations, and split verification mechanisms based on digital signatures and homomorphic hashes, RFL aims to provide privacy and verifiability guarantees while keeping per-client computation and communication overhead independent of the global model dimension. This positions RFL as a complementary direction to prior federated edge computing frameworks that focus primarily on performance or standard privacy, rather than on server-side inference attacks and verifiable aggregation in edge settings.

IX. Conclusion

We have investigated the Identity Forgery and Gradient Inversion Attack (IFGIA) against privacy-preserving federated learning schemes for edge computing, with a particular focus on new client integration phases. Our analysis and experiments show that IFGIA can achieve success rates above 99.5% against several state-of-the-art secure-aggregation-based schemes, enabling a malicious central server to reconstruct local gradients and infer sensitive client data.

To mitigate this threat, we proposed the Robust Federated Learning (RFL) framework, which combines model splitting, lightweight differential privacy on intermediate representations, and split verification based on digital signatures and homomorphic hashes. RFL provides global privacy guarantees, reduces the success rate of IFGIA to 73.6%, verifies client identities and aggregation integrity, and decreases the communication overhead per client from 375 MB to 40 KB while speeding up edge-side training by at least tenfold in our experiments.

In future work, we plan to extend RFL to more complex model architectures such as transformers, and to investigate stronger robustness against poisoning attacks and collusion between the central server and a subset of edge servers.

References

[1] X. Lin, J. Wu, J. Li, X. Zheng, and G. Li, "Friend-as-learner: Socially-driven trustworthy and efficient wireless federated edge learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 269–283, Jan. 2023.

[2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, 2019, Art. no. 12.

[3] Y. Wang, Y. Xu, Q. Shi, and T.-H. Chang, "Quantized federated learning under transmission delay and outage constraints," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 323–341, Jan. 2022.

[4] Y. Wang, Q. Shi, and T.-H. Chang, "Why batch normalization damage federated learning on non-IID data?," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 1692–1706, Jan. 2025.

[5] J. Zhang et al., "FLPurifier: Backdoor defense in federated learning via decoupled contrastive training," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4752–4766, 2024, doi: 10.1109/TIFS.2024.3384846.

[6] J. Zhang et al., "BadCleaner: Defending backdoor attacks in federated learning via attention-based multi-teacher distillation," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4559–4573, Sep./Oct. 2024, doi: 10.1109/TDSC.2024.3354049.

[7] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun.*, I. Ray, N. Li, and C. Kruegel, Eds., 2015, pp. 1322–1333.

[8] N. Bugshan, I. Khalil, M. S. Rahman, M. Atiquzzaman, X. Yi, and S. Badsha, "Toward trustworthy and privacy-preserving federated deep learning service framework for industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1535–1547, Feb. 2023.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.

[9] Y. Li, J. Li, and Y. Wang, "Privacy-Preserving spatiotemporal scenario generation of renewable energies: A federated deep generative learning approach," *IEEE Trans. Ind. Informat.*, vol. 18, no. 4, pp. 2310–2320, Apr. 2022.
[10] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-Preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr. 2022.
[11] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
[12] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–36, 2022.
[13] J. Gao et al., "Secure aggregation is insecure: Category inference attack on federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 147–160, Jan./Feb. 2023.
[14] X. Zhang, C. Chen, Y. Xie, X. Chen, J. Zhang, and Y. Xiang, "A survey on privacy inference attacks and defenses in cloud-based deep neural network," *Comput. Stand. Interfaces*, vol. 83, 2023, Art. no. 103672.
[15] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *Proc. ACM SIGSAC*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds., 2022, pp. 2429–2443.
[16] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3/4, pp. 211–407, 2014.
[17] A. Blum, C. Dwork, F. McSherry, and K. Nissim, "Practical privacy: The SuLQ framework," in *Proc. 24th ACM SIGMOD-SIGACT-SIGART Symp. Princ. Database Syst.*, 2005, pp. 128–138.
[18] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020.
[19] H. Zhou, G. Yang, H. Dai, and G. Liu, "PFLF: Privacy-preserving federated learning framework for edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1905–1918, 2022.
[20] X. Li, H. Yan, Z. Cheng, W. Sun, and H. Li, "Protecting regression models with personalized local differential privacy," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 960–974, Mar./Apr. 2023.
[21] X. Sun, P. Zhang, J. K. Liu, J. Yu, and W. Xie, "Private machine learning classification based on fully homomorphic encryption," *IEEE Trans. Emerg. Topics Comput.*, vol. 8, no. 2, pp. 352–364, Apr.–Jun. 2020.
[22] Q. Zhang, C. Xin, and H. Wu, "GALA: Greedy computation for linear algebra in privacy-preserved neural networks," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–16.
[23] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-Preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.
[24] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, San Jose, CA, USA, 2017, pp. 19–38.
[25] P. Mohassel and P Rindal, "ABY^3: A mixed protocol framework for machine learning," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, ACM, 2018, pp. 35–52.
[26] R. Xu, N. Baracaldo, Y. Zhou, A. Anwar, and H. Ludwig, "HybridAlpha: An efficient approach for privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 13–23.
[27] Q. Duan, S. Hu, R. Deng, and Z. Lu, "Combined federated and split learning in edge computing for ubiquitous intelligence in Internet of Things: State-of-the-art and future directions," *Sensors*, vol. 22, no. 16, 2022, Art. no. 5983.
[28] Y. Gao et al., "Evaluation and optimization of distributed machine learning techniques for Internet of Things," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2538–2552, 2022.
[29] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2023.
[30] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 1736–1751, 2021.
[31] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.
[32] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
[33] G. George, A. Lozano, and M. Haenggi, "Distribution of the number of users per base station in cellular networks," *IEEE Wireless Commun. Lett.*, vol. 8, no. 2, pp. 520–523, Apr. 2019.
[34] B. Pilanawithana, S. Atapattu, and J. S. Evans, "Distribution of number of users per cell in a poisson wireless network with shadowing," in *Proc. 2016 Australian Commun. Theory Workshop*, 2016, pp. 153–156.
[35] X. Chang, T. Oda, E. Spaho, M. Ikeda, L. Barolli, and F. Xhafa, "Performance analysis of WMNs using hill climbing algorithm considering normal and uniform distribution of mesh clients," in *Proc. Conf. Complex, Intell., Softw. Intensive Syst.*, 2013, pp. 424–427.
[36] Y. Wang, W. Fu, and D. P. Agrawal, "Gaussian versus uniform distribution for intrusion detection in wireless sensor networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 2, pp. 342–355, Feb. 2013.
[37] H. Li, D. Liu, Y. Dai, and T. H. Luan, "Engineering searchable encryption of mobile cloud networks: When QoE meets QoP," *IEEE Wireless Commun.*, vol. 22, no. 4, pp. 74–80, Aug. 2015.
[38] H. Zhou, G. Yang, Y. Huang, H. Dai, and Y. Xiang, "Privacy-Preserving and verifiable federated learning framework for edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 565–580, 2023.
[39] J. Zhang, Y. Liu, D. Wu, S. Lou, B. Chen, and S. Yu, "VPFL: A verifiable privacy-preserving federated learning scheme for edge computing systems," *Digit. Commun. Netw.*, vol. 9, no. 4, pp. 981–989, 2022.
[40] Q. Zeng, Y. Du, K. Huang, and K. K. Leung, "Energy-Efficient radio resource allocation for federated edge learning," in *Proc. IEEE Int. Conf. Commun. Workshops*, Dublin, Ireland, 2020, pp. 1–6.
[41] S. Awan, F. Li, B. Luo, and M. Liu, "Poster: A reliable and accountable privacy-preserving federated learning framework using the blockchain," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 2561–2563.
[42] H. Tian, Y. Wen, F. Zhang, Y. Shao, and B. Li, "A distributed threshold additive homomorphic encryption for federated learning with dropout resiliency based on lattice," in *Proc. Int. Symp. Cyberspace Saf. Secur.*, X. Chen, J. Shen, and W. Susilo, Eds., 2022, vol. 13547, pp. 277–292.
[43] R. Zuo, H. Tian, Z. An, and F. Zhang, "Post-Quantum privacy-preserving aggregation in federated learning based on lattice," in *Proc. Int. Symp. Cyberspace Saf. Secur.*, X. Chen, J. Shen, and W. Susilo, Eds., 2022, vol. 13547, pp. 314–326.
[44] A. R. Chowdhury, C. Guo, S. Jha, and L. van der Maaten, "EIFFfeL: Ensuring integrity for federated learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2535–2549.
[45] J. Guo, J. Wu, A. Liu, and N. N. Xiong, "LightFed: An efficient and secure federated edge learning system on model splitting," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2701–2713, Nov. 2022.
[46] Y. Guo, F. Liu, T. Zhou, Z. Cai, and N. Xiao, "Privacy vs. efficiency: Achieving both through adaptive hierarchical federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1331–1342, Apr. 2023.
[47] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.



Hao Zhou (Member, IEEE) received the PhD degree from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2024. He is currently an associate professor with the College of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing. His research interests include privacy-preserving machine learning and information security.

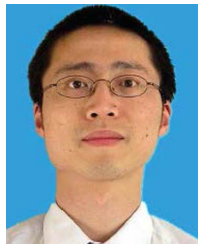


Hua Dai (Member, IEEE) received the PhD degree in computer science from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2011. He is currently a professor with the Nanjing University of Posts and Telecommunications, China. His research interests include data management and security, and cloud computing security.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.



Geng Yang received the PhD degree in computational mathematics from Laval University in 1994. From 1994 to 1996, he was a postdoctoral research fellow with the Center for Research on Computation and Its Applications, University of Montreal, Canada. His research interests include information security, cloud computing security and privacy-preserving, and parallel computing. He is a member of IEEE Computer Society.



Yang Xiang (Fellow, IEEE) received the PhD degree in computer science from Deakin University, Melbourne, VIC, Australia, in 2007. He is currently with the Swinburne University of Technology, Australia. His research interests include network and system security, and wireless systems. He has been a PC member for many international conferences, such as IEEE ICC, IEEE GLOBECOM, SECRYPT, Malware, and IEEE ICPADS. He has served as reviewer for many international journals, such as *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Communications Letters*, and *IEEE Journal on Selected Areas in Communications*. He is on the editorial board of *Journal of Network and Computer Applications*.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:26:27 UTC from IEEE Xplore. Restrictions apply.